
COMPARATIVE STUDY OF END-TO-END VIRTUAL MACHINE CONNECTIVITY ACROSS SUBNETS USING SSHUTTLE, WIREGUARD, AND STATIC ROUTING

Visesh Bentula

Department of Computer Science
Texas A&M University-San Antonio
vbent01@jaguar.tamu.edu

ABSTRACT

This paper presents a self-adaptive virtual network system for enabling end-to-end communication between virtual machines across routed subnets on the FABRIC testbed [1, 2]. In Milestone 1, a 3-node topology was built on FABRIC and validated using manual static routing, following standard IP forwarding principles [3] and adapting the official `static_routes_sshuttle` recipe [4]. In Milestone 2, the same topology was replicated across three environments FABRIC, OMNeT++, and ns-3 [1, 5, 6, 7] to evaluate and compare three routing/tunneling techniques: `sshuttle` (TCP-based proxy) [8, 9, 10], WireGuard (encrypted L3 overlay) [11, 12], and traditional static IP routing [3]. A detailed analysis revealed tradeoffs in each method under varying traffic and fault conditions. Based on this insight, Milestone 3 introduces an Auto Controller that dynamically monitors real-time metrics such as latency, packet loss, and tunnel health in FABRIC using `FABlib` and common measurement tools (e.g., `iperf3`, `nc`) [2, 13, 14], and automatically switches between `sshuttle`, WireGuard, and static routing to maintain connectivity. The system also includes fault injection and recovery logic for robustness under failure. Unlike prior work, this solution spans both simulation and emulation environments [7, 5] and culminates in a fully adaptive, fabric-aware routing orchestrator.

1 Introduction

Reliable end-to-end connectivity across routed subnets is a fundamental requirement for modern cloud, research, and testbed environments [3]. In many practical scenarios, virtual machines (VMs) are deployed on separate Layer-3 networks for isolation, yet still need to exchange monitoring data, control traffic, or user applications with predictable performance and, in some cases, strong security guarantees. Simply “making pings work” is not enough: operators must choose between plain IP routing, encrypted VPN tunnels, and application-level proxies, each of which behaves differently under changing network conditions, failures, and traffic types [11, 8, 12].

In this project, I focus on a minimal but representative three-node topology where two end hosts (Node1 and Node3) reside on different subnets, and a middle node (Node2) has interfaces to both networks, following the general pattern of the `static_routes_sshuttle` recipe [4]. The experiments are deployed on the FABRIC testbed, which provides sliceable Linux VMs, programmable Layer-2 networks, and full root access for installing custom tools [1, 2]. On top of this topology, I implement three connectivity methods: (1) manual static routing, where Node2 acts as a traditional router using `ip route` and IP forwarding [3]; (2) an encrypted Layer-3 overlay using WireGuard, where all three nodes participate in a 10.0.0.0/24 tunnel [11, 12]; and (3) `sshuttle`, a user-space TCP proxy that forwards application traffic over SSH without changing the routing tables on every VM [8, 9, 10]. These three mechanisms span a spectrum from simple and transparent (static) to secure (WireGuard) to very convenient but TCP-only (`sshuttle`).

To better understand how these approaches behave beyond a single testbed, I replicated the same logical topology in two simulation environments: OMNeT++ with the INET framework and the ns-3 discrete-event simulator [5, 6, 7]. In OMNeT++, I configured static routing, a TCP relay path, and a WireGuard-style overlay using UDP/TCP applications and IP forwarding modules to mimic the real FABRIC setup. In ns-3, I built an equivalent three-node scenario with point-to-point links and implemented three modes: pure static routing, a “WireGuard-style” overlay link between Node1

and Node3, and an `sshuttle`-like TCP relay via Node2, instrumented with `FlowMonitor` and `NetAnim` for throughput, delay, and packet-loss measurements [7]. Using `FABRIC` plus `OMNeT++` and `ns-3` allows me to cross-check behavior, visualize protocol flows, and compare performance trends across real and simulated environments.

The work evolved over three milestones. In Milestone 1, I created the three-node `FABRIC` slice, configured static routing, and validated basic connectivity and routing tables as a baseline [1, 2, 3]. In Milestone 2, I extended this topology to all three tools (`sshuttle`, `WireGuard`, and static routing) and mirrored the design in `OMNeT++` and `ns-3`, collecting screenshots, flow statistics, and qualitative observations [8, 11, 7, 5]. A key insight from this stage was that no single method is “best” in all situations: static routing offers the lowest latency and full protocol support but no encryption, `WireGuard` adds strong security at the cost of tunnel setup and overhead, and `sshuttle` is extremely convenient for TCP but breaks ICMP and UDP-based workflows [12, 3]. This observation led to the intuitive idea that the network should not pick one tool statically, but instead adapt its tunneling strategy based on current requirements and link health.

Milestone 3 turns this intuition into a self-adaptive system running entirely on `FABRIC`. I implemented an Auto Controller that periodically measures latency and packet loss between Node1 and Node3, monitors tunnel health, and then automatically selects between static routing, `WireGuard`, and `sshuttle` according to simple policy inputs (e.g., “encryption required” or “ICMP must work”) [2]. The controller can also inject faults (e.g., dropping routes or stopping the VPN) and verify that it recovers by switching to an alternative mode. Continuous `iperf3` traffic is used to emulate real load while the controller cycles through measurement, decision, reconfiguration, and verification [13, 14].

Overall, this project is not just a side-by-side comparison of three tools, but a complete lifecycle: starting from a basic connectivity problem, exploring multiple routing and tunneling options across both real and simulated platforms, and finally integrating them into a unified, autonomous tunnel-switching framework that maintains end-to-end VM connectivity across subnets in real time [1, 7, 5].

2 Methodology

This project follows an experimental methodology that combines a real federated testbed (`FABRIC`) with two discrete-event simulators (`ns-3` and `OMNeT++`) to study and automate end-to-end connectivity across routed subnets [1, 7, 5, 6]. We focus on three concrete mechanisms: `sshuttle` (TCP-based proxy), `WireGuard` (encrypted L3 overlay), and conventional static IP routing [8, 11, 3, 4]. The methodology has three main phases: (i) building a reproducible three-node topology, (ii) implementing and validating the three tunneling/routing approaches in `FABRIC`, `ns-3`, and `OMNeT++`, and (iii) designing an autonomous controller in Milestone 3 that monitors live network conditions using `FABlib` and common measurement tools and automatically switches between methods based on simple decision rules [2, 13, 14].

2.1 Experimental Topology on FABRIC

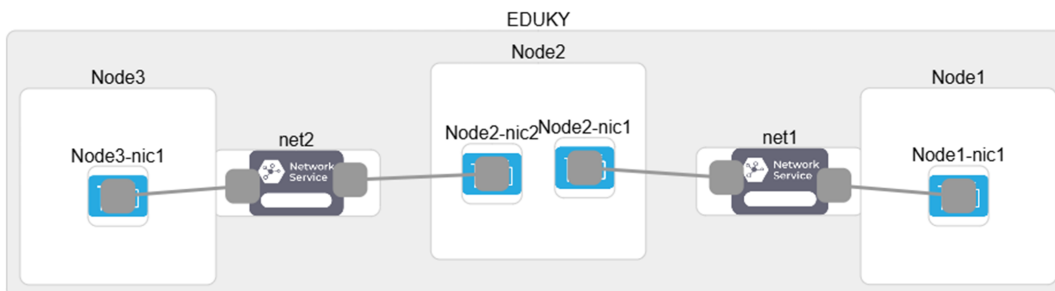


Figure 1: Experimental three-node topology on the `FABRIC` testbed. Node1 and Node3 are placed on two disjoint Layer-2 networks (`net1` and `net2`), while Node2 resides at the `EDUKY` site with two interfaces (`Node2-nic1` and `Node2-nic2`) and acts as the intermediate relay/router. All three nodes also have a separate management interface (not shown) for slice access. This topology is used throughout the project to implement and compare `sshuttle`, `WireGuard`, and static routing for end-to-end connectivity between Node1 and Node3.

All physical experiments were conducted on the `FABRIC` testbed using three Ubuntu Linux virtual machines arranged in a linear chain: Node1 → Node2 → Node3. Two Layer-2 broadcast domains were created:

- **LAN (`net1`):** connects Node1 and Node2, configured as `192.168.1.0/24`.

- **WAN (net2):** connects Node2 and Node3, configured as 192.168.2.0/24.

Node1 and Node3 each have a single interface in their respective subnet, while Node2 has two interfaces and acts as the potential router or relay between the two networks. Management access is provided via a separate FABRIC management network; however, all measurements reported in this paper are taken only on the experimental subnets. In the initial baseline configuration, only adjacent nodes can communicate (Node1 ↔ Node2 and Node2 ↔ Node3), whereas Node1 and Node3 cannot reach each other until one of the proposed methods is enabled.

2.2 Tunneling and Routing Methods on FABRIC

Three connectivity approaches were implemented on the above topology. Each method was configured, verified using routing tables and connectivity tests, and then used as a building block for the Milestone 3 controller.

2.2.1 Dynamic SSH tunneling with sshuttle

The first method uses `sshuttle`, a user-space VPN-like tool that transparently forwards TCP traffic over an SSH connection. `sshuttle` is launched on Node1, establishing an SSH-based tunnel to Node2 and capturing any TCP traffic destined for the remote subnet 192.168.2.0/24. On Node1, a dummy route (192.168.2.0/24 via 127.0.0.1) is installed so that the Linux kernel emits packets for the otherwise unreachable subnet; these packets are then intercepted by `iptables` and relayed through the SSH tunnel to Node2, which forwards them to Node3.

No configuration changes are required on Node3. Connectivity is validated using a TCP probe (e.g., `nc -zv 192.168.2.2 22`) from Node1 to Node3's SSH port, which succeeds once the tunnel is active. As a limitation, `sshuttle` forwards only TCP traffic; ICMP and UDP are not supported, which motivated the use of additional methods for full-protocol experiments.

2.2.2 WireGuard encrypted L3 overlay

The second method uses WireGuard to provide an encrypted Layer-3 VPN overlay across the same physical topology. WireGuard is installed on all three nodes, and each node is configured with a `wg0` interface and an overlay IP address:

- Node1: 10.0.0.1/24
- Node2: 10.0.0.2/24
- Node3: 10.0.0.3/24

Public/private key pairs are generated on each node, and static peer configurations are created so that Node2 maintains WireGuard sessions with both Node1 and Node3. IP forwarding is enabled on Node2, and `iptables` rules are installed to allow forwarding and NAT between the WireGuard overlay and the underlay subnets. Once activated via `wg-quick up wg0`, Node1 and Node3 can successfully ping each other's overlay addresses and underlay addresses across the tunnel (e.g., Node1 pinging 10.0.0.3 and 192.168.2.2).

WireGuard statistics (`wg show`) are used to confirm non-zero byte counters for each peer, verifying that encrypted packets are exchanged through the tunnel. Unlike `sshuttle`, this method supports all IP protocols (TCP, UDP, ICMP) and provides end-to-end encryption at the cost of slightly more initial configuration.

2.2.3 Static routing baseline

The third method uses manual static routes with Node2 acting as a conventional router between the two subnets. Any leftover `sshuttle` or WireGuard configurations are removed to avoid interference. IP forwarding is enabled on Node2, and explicit static routes are added on the edge nodes:

- On Node1: route to 192.168.2.0/24 via Node2's 192.168.1.2 interface.
- On Node3: route to 192.168.1.0/24 via Node2's 192.168.2.1 interface.

After these entries are installed, Node1 and Node3 can ping each other's underlay IPs (192.168.1.1 and 192.168.2.2) with zero packet loss and sub-millisecond latency, and TCP connectivity to port 22 is again verified using `nc`. Because this approach involves no tunneling or encryption, it serves as a transparent baseline for performance comparisons and for supporting protocols that `sshuttle` cannot carry.

2.3 Simulation Methodology in ns-3 and OMNeT++

To complement the FABRIC experiments and allow controlled performance studies, the same three-node topology is recreated in two simulators: ns-3 and OMNeT++. In both cases, Node1 (Shore) and Node3 (Ship) are placed on separate subnets, with Node2 (Relay) providing the intermediate forwarding or relay function.

2.3.1 ns-3 setup

In ns-3, three nodes are connected using two point-to-point links that emulate the 192.168.1.0/24 and 192.168.2.0/24 networks [7]. Different configurations are implemented to model each method:

- **Static routing:** `Ipv4StaticRoutingHelper` is used to manually add routes such that traffic between Node1 and Node3 passes through Node2, mirroring the Linux `ip route` configuration on FABRIC.
- **WireGuard-style overlay:** an additional logical link with an overlay subnet (e.g., 10.0.0.0/24) is created directly between Node1 and Node3 to mimic an encrypted tunnel, while the underlay links still exist but do not carry user traffic.
- **sshuttle-style relay:** a TCP client/server application is configured with Node2 forwarding traffic, approximating the behavior of a TCP-level relay where Node2 acts as a middlebox rather than a pure router.

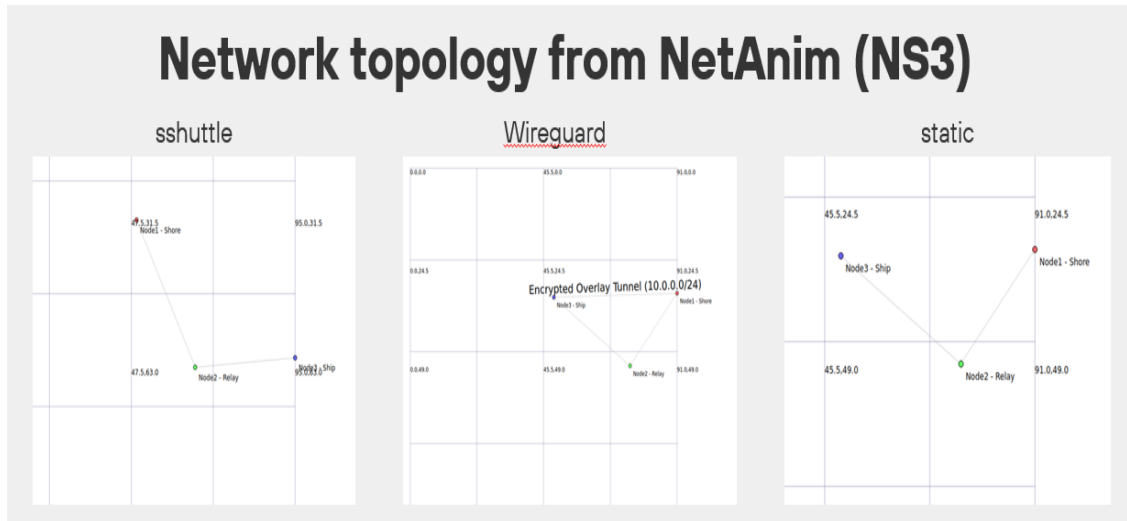


Figure 2: Network topology visualized in NetAnim for the ns-3 experiments. Each panel shows the same three-node layout (Node1, Node2, Node3) under a different connectivity method: `sshuttle`-style TCP relay (left), WireGuard-style encrypted overlay tunnel (center), and pure static IP routing (right). The positions are identical, but the logical paths and link labels reflect the active routing or tunneling configuration in each case.

An ns-3 `FlowMonitor` instance is attached to all nodes to capture per-flow statistics such as throughput, end-to-end delay, packet delivery ratio, and jitter under identical traffic patterns [7]. By exporting these traces, it becomes possible to compare, for example, how much additional delay the `sshuttle`-style relay introduces relative to pure static routing, or how the WireGuard-style overlay behaves when link characteristics (e.g., data rate or propagation delay) are varied.

In addition to `FlowMonitor`, the simulation can be visualized using NetAnim or PCAP traces, which helps confirm that packets follow the expected paths (Node1 $\rightarrow$ Node2 $\rightarrow$ Node3 for static routing and relay mode, and Node1 $\rightarrow$ Node3 directly for the overlay). This cross-checking ensures that the ns-3 scenarios faithfully reproduce the logical behavior of the FABRIC experiments before any quantitative comparison is made.

2.3.2 OMNeT++ setup

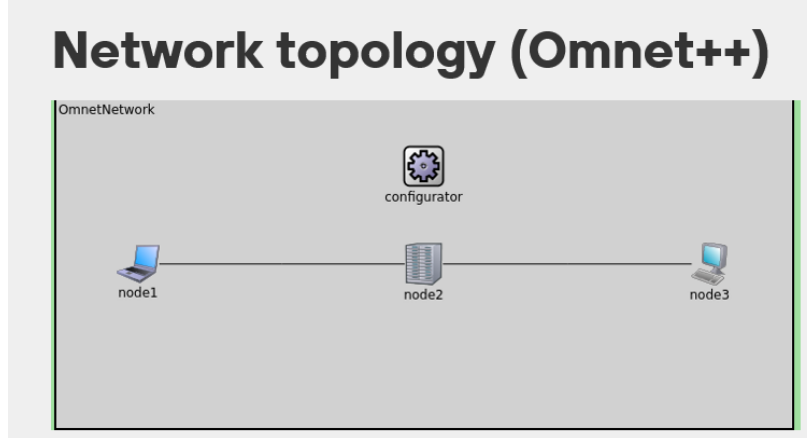


Figure 3: OMNeT++ three-node topology mirroring the FABRIC experiment. Node1 and Node3 reside on separate subnets, while Node2 has two interfaces and forwards traffic between them. This setup is used to emulate `sshuttle`-style relays, WireGuard-like overlays, and static routing.

In OMNeT++, the same three-node topology is built using INET models. Node2 is configured with two interfaces and IP forwarding enabled, while Node1 and Node3 each have a single interface in their respective subnet. Different configurations are used to emulate each approach:

- **Static routing:** manual entries are provided in the `.ini` file routing tables, mirroring the FABRIC static routes.
- **sshuttle-style relay:** TCP client/server applications (e.g., `TcpBasicClientApp` and `TcpSinkApp`) are deployed with Node2 forwarding packets between the two subnets.
- **WireGuard-style overlay:** a virtual overlay or tunnel is modeled; routing misconfiguration in early runs (ICMP unreachable warnings) is used to study failure behavior and debug missing overlay routes.

Sequence charts and packet traces from OMNeT++ are used to verify the correctness of ARP resolution, TCP three-way handshakes, and UDP data exchange across Node2.

2.4 Autonomous Tunnel Controller (Milestone 3)

Building on the validated FABRIC configurations, Milestone 3 introduces a Python-based controller that automatically discovers the slice topology, orchestrates the three connectivity modes, and makes run-time decisions based on measured performance.

2.4.1 Slice discovery and IP mapping

Using the FABlib API, the controller retrieves the existing slice, enumerates all nodes, and classifies them into *left*, *router*, and *right* roles based on the number and subnet of their interfaces. It then extracts the IP addresses of each interface and automatically derives the corresponding /24 subnets (e.g., `192.168.1.0/24` and `192.168.2.0/24`). This removes any hard-coded assumptions about node names or addresses and makes the controller reusable on future slices.

2.4.2 Mode activation and background traffic

For each method, the controller defines dedicated activation and teardown functions:

- `use_static()` enables IP forwarding on Node2 and installs the appropriate static routes on Node1 and Node3.
- `use_wireguard()` brings down any old `wg0` interfaces and then brings them back up on all nodes using pre-created configuration files.

- `use_sshuttle()` first enables static routing as an underlay, then starts `sshuttle` on Node1 in daemon mode, tunneling the remote subnet via Node2.

A long-running `iperf3` flow from Node1 to Node3 is started at the beginning of each experiment to generate realistic background traffic while the controller is making decisions. Corresponding cleanup functions stop the traffic generator and remove routes or tunnels when needed.

2.4.3 Monitoring, metrics, and decision logic

Every control cycle (e.g., every 20 s), the controller measures connectivity from Node1 to Node3 using a short ping burst. From the output it parses the average round-trip latency and packet loss percentage. If ping fails or reports 100% loss, the controller considers the link unhealthy. These metrics, along with simple scenario flags (“encryption required” and “ICMP required”), are passed to a decision function that selects the preferred mode:

- If encryption is required and WireGuard is healthy, stay on or switch to WireGuard.
- If ICMP and low-latency are prioritized, prefer static routing whenever it yields low delay and loss.
- For TCP-only scenarios without encryption, prefer `sshuttle` when loss and latency are acceptable.
- If the current mode exhibits high loss (> 50%) or very high latency, fail over to another method (typically static routing) to restore connectivity.

Mode-switching is implemented carefully so that only the necessary components are stopped and the newly selected mode is activated with minimal disruption. Each cycle is logged with the chosen mode, measured latency, and loss to create a compact history for later analysis.

2.4.4 Fault injection and scenarios

To evaluate robustness, the controller periodically injects artificial faults into the currently active method (e.g., bringing down `wg0`, deleting static routes, or killing the `sshuttle` process). After each fault, the controller observes the resulting metrics and, if necessary, automatically switches to a healthier mode.

Three high-level scenarios are executed:

1. **Secure scenario:** encryption and ICMP are both required, so WireGuard is preferred; when a WireGuard fault is injected, the controller temporarily falls back to static routing and then returns to WireGuard once the tunnel is restored.

```
print("\n=== RUNNING: SECURE SCENARIO (Encryption + ICMP required) ===")

hist_secure = autopilot(
    interval=20,
    iterations=10,
    need_encryption=True,
    need_icmp=True,
    demo_fault_every=3
)

print_history(hist_secure, "SECURE SCENARIO")
```

Figure 4: Python controller invocation for the *Secure scenario*. The script configures the `autopilot` function with both encryption and ICMP required, runs for multiple control iterations, and logs the mode history as the controller switches between WireGuard and static routing after injected tunnel faults.

2. **ICMP-priority scenario:** low latency and reliable ICMP are prioritized, so static routing is preferred; if static routing is broken, the controller temporarily uses `sshuttle` before switching back.

```

print("\n=== RUNNING: ICMP PRIORITY SCENARIO (Static preferred) ===")
hist_icmp = autopilot(
    interval=20,
    iterations=10,
    need_encryption=False,
    need_icmp=True,
    demo_fault_every=3
)
print_history(hist_icmp, "ICMP PRIORITY SCENARIO")

```

Figure 5: Python controller invocation for the *ICMP-priority scenario*. Here the autopilot function is configured to prioritize ICMP and low latency without encryption, causing static routing to be selected as the primary mode while the controller falls back to `sshuttle` when static routes are deliberately broken.

3. **TCP-only scenario:** no ICMP requirement and no encryption requirement; `sshuttle` remains the dominant mode with static routing acting as a resilient underlay.

```

print("\n=== RUNNING: TCP-ONLY SCENARIO (sshuttle preferred) ===")
hist_tcp = autopilot(
    interval=20,
    iterations=10,
    need_encryption=False,
    need_icmp=False,
    demo_fault_every=3
)
print_history(hist_tcp, "TCP-ONLY SCENARIO")

```

Figure 6: Python controller invocation for the *TCP-only scenario*. In this configuration the autopilot function disables both encryption and ICMP requirements, so `sshuttle` is selected as the preferred mode while static routing remains available as a resilient underlay during injected faults.

This methodology produces a full lifecycle from topology discovery and baseline configuration (Milestone 1), through comparative evaluation in testbed and simulators (Milestone 2), to an automated, self-adjusting tunnel-selection system (Milestone 3) that reacts to live network conditions.

3 Results and Discussion

This section summarizes the experimental results obtained from the FABRIC testbed, the ns-3 and OMNeT++ simulations, and the Milestone 3 autonomous controller. The focus is on (i) validating end-to-end connectivity for each method, (ii) comparing performance and behavior across environments, and (iii) evaluating how well the controller selects and switches between modes under different scenarios. For each platform, we report both qualitative observations (e.g., successful handshakes, routing behavior, failure modes) and quantitative metrics such as latency, packet loss, and throughput where applicable. We also highlight discrepancies between the real testbed and the simulators, explaining when they arise from modeling simplifications versus configuration issues. Finally, we discuss the implications of these results for designing self-adaptive connectivity mechanisms on larger, more complex programmable infrastructures, where multiple tunnels and routing options may coexist.

3.1 Connectivity Results on FABRIC

3.1.1 `sshuttle`

On FABRIC, `sshuttle` [8, 9, 10] successfully established an SSH-based TCP tunnel from Node1 to Node3 via Node2 using the `-method=nat` option on a 3-node slice configured with FABlib on the FABRIC testbed [1, 2, 4]. TCP connectivity to Node3’s SSH port (`nc -zv 192.168.2.2 22`) was consistently successful, confirming that TCP flows were transparently redirected through the tunnel without manually modifying the routing tables on Node1 or Node3; `nc` itself is a lightweight TCP/UDP testing tool [14]. Once the tunnel was active, repeated connection attempts showed stable behavior across multiple runs, which indicates that `sshuttle` can provide reliable end-to-end reachability for typical remote shell and application traffic.

However, ICMP echo requests (ping) and UDP traffic did not traverse the tunnel, matching sshuttle’s design as a TCP-only forwarder [8]. This limitation makes sshuttle ideal for quick debugging or remote shell access, but less suitable as a general solution for full-protocol experiments that require, for example, ICMP-based monitoring or UDP-based applications. In practice, this means that sshuttle works well as a convenient, low-friction tool during slice bring-up and troubleshooting, but needs to be complemented by other mechanisms when more complete connectivity is required.

3.1.2 WireGuard

With WireGuard, all three nodes were equipped with a wg0 interface and overlay IPs in the 10.0.0.0/24 subnet. After exchanging keys and bringing up the interfaces, Node1 could successfully ping Node3’s WireGuard IP (10.0.0.3) with zero packet loss and sub-millisecond latency, and nc confirmed TCP connectivity to the SSH service on Node3. These results demonstrate that once the overlay is correctly configured, WireGuard behaves almost like a direct Layer-3 link between the endpoints, while still providing encryption [11, 12].

Initially, one asymmetry was observed: Node3 could not reach Node1’s underlay address (192.168.1.1) due to missing AllowedIPs entries and forwarding/NAT rules on Node2. This misconfiguration produced one-way reachability until the rules were corrected. Once these forwarding rules were fixed, WireGuard provided protocol-agnostic, encrypted end-to-end connectivity with minimal overhead, demonstrating that most of the complexity lies in configuration rather than runtime performance [12]. The experiment also highlights that forgetting a single route or AllowedIPs entry can lead to subtle connectivity issues, which motivates automation through the controller developed in Milestone 3.

3.1.3 Static Routing

Static routing on FABRIC, with Node2 acting as a classic IP router, delivered clean, low-latency connectivity between the two subnets. After enabling IP forwarding on Node2 and installing explicit routes on Node1 and Node3 following standard static-routing practice [3], bidirectional ping between 192.168.1.1 and 192.168.2.2 was successful, and the routing tables clearly showed traffic being forwarded via Node2. In repeated tests, round-trip times remained consistently low and packet loss was effectively zero, confirming that the baseline routed path is both stable and predictable.

As expected, this approach provided a protocol-complete path without encryption. The main drawbacks are the lack of security and the need to manually update routes when the topology changes or when additional subnets are added [3]. In this project, static routing primarily serves as a baseline for measuring the performance cost of sshuttle and WireGuard and as a reliable “fallback” method for the controller. Because it is simple and transparent, static routing is also useful for validating that the underlying FABRIC links and IP addressing are correct before enabling more advanced tunneling mechanisms.

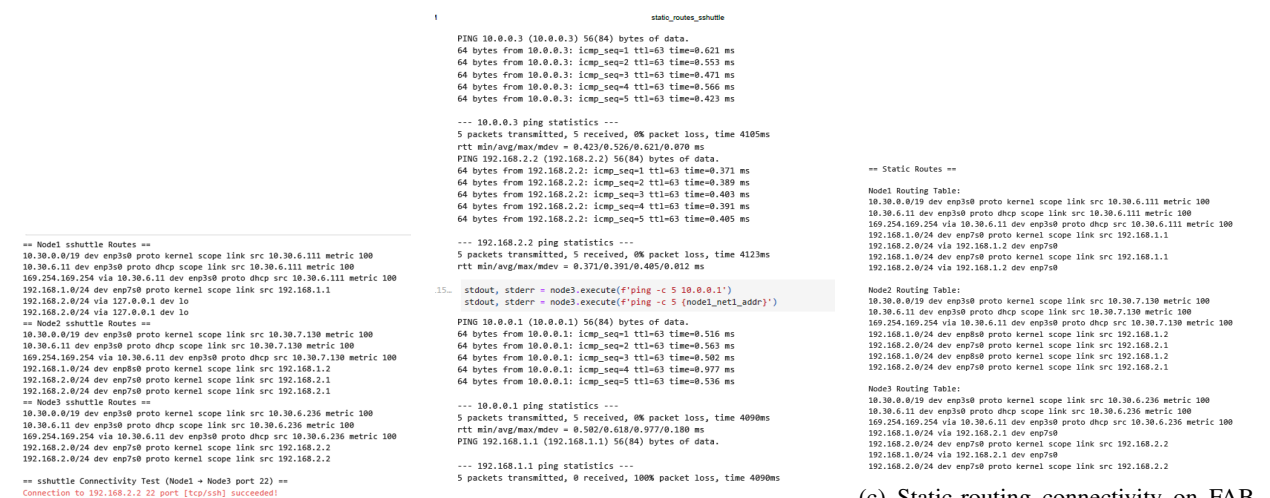


Figure 7: Connectivity verification for the three methods on the FABRIC testbed: (a) sshuttle TCP tunnel, (b) WireGuard encrypted overlay, and (c) static IP routing via Node2.

3.2 Simulation Results in ns-3

In ns-3, the same three-node topology and three methods were recreated to study performance under controlled conditions.

For the **sshuttle-style relay**, Node2 acted as a TCP proxy using an `OnOffApplication` and sink configuration. FlowMonitor results showed successful bidirectional TCP transfer with consistent throughput around ≈ 2.2 Mbps and expected delay introduced by the intermediate relay. No packet loss or routing errors were observed, though end-to-end delay was higher than for direct tunnels due to the extra processing at Node2.

For the **WireGuard-style overlay**, a direct overlay link between Node1 and Node3 in the `10.0.0.0/24` subnet bypassed Node2 entirely. Packets experienced very low delay (on the order of ≈ 0.007 s) and zero loss, with FlowMonitor reporting stable throughput around ≈ 0.010 Mbps. This configuration approximates an ideal encrypted tunnel with negligible overhead in the simulator (cryptographic costs were not explicitly modeled).

For **static routing**, Node2 was configured as a router between `192.168.1.0/24` and `192.168.2.0/24`, replicating the FABRIC design. The simulation produced zero packet loss, stable throughput (≈ 0.010 Mbps), and minimal delay (≈ 0.028 s), validating that the static-routing baseline is both robust and predictable in ns-3.

Overall, ns-3 results align with the FABRIC behavior: static routing provides the cleanest baseline, the WireGuard-style tunnel offers a low-latency overlay, and the sshuttle-style relay introduces additional delay but maintains reliable TCP connectivity.

3.3 Simulation Results in OMNeT++

In OMNeT++, results were examined using sequence charts and packet traces.

For the **sshuttle-style configuration**, a TCP client on Node1 connected to a TCP sink on Node3, with Node2 forwarding packets between subnets. Sequence charts showed ARP resolution on both links, followed by a normal three-way TCP handshake and data transfer without retransmissions or resets, indicating a stable, loss-free relay connection across Node2.

For **static routing**, UDP client/server applications were used. ARP exchanges correctly established link-layer mappings, and UDP packets flowed from Node1 to Node3 and back through Node2. The absence of packet loss or retransmissions in the traces confirmed that the static routes and IP forwarding behaved as expected, mirroring the FABRIC and ns-3 baselines.

For the **WireGuard-style overlay**, initial attempts resulted in ICMP `destination unreachable` messages for packets destined to the overlay subnet (`10.0.1.0/24`). This indicated that the virtual tunnel interfaces or overlay routes were not fully configured by the INET IPv4 configurator. While UDP applications did initialize, the lack of valid overlay routes prevented end-to-end traffic, highlighting the importance of carefully modeling tunnel interfaces in OMNeT++ when emulating VPN behavior.

3.4 Autonomous Controller Scenarios (Milestone 3)

The Milestone 3 controller was evaluated under three main scenarios, each running for multiple control cycles with continuous background traffic and periodic fault injection.

3.4.1 Secure scenario (WireGuard preferred)

In the secure scenario, both encryption and ICMP support were required, so WireGuard was the preferred mode. When WireGuard was healthy, the controller selected and stayed on this mode. Whenever a fault was injected (e.g., bringing down `wg0`), the controller observed degraded metrics (high loss or failed ping) and automatically switched to static routing to restore connectivity. Once WireGuard was reactivated and metrics improved, the controller switched back, producing a repeating pattern of *WireGuard* $\rightarrow$ *Fault* $\rightarrow$ *Static* $\rightarrow$ *WireGuard*.

3.4.2 ICMP-priority scenario (Static preferred)

In the ICMP-priority scenario, low-latency ping and reliable ICMP were prioritized. The controller therefore started in static-routing mode, which consistently delivered very low latency (on the order of 0.2–0.3 ms) and zero loss on FABRIC. When static routing was intentionally broken (e.g., disabling IP forwarding or deleting routes), the decision engine temporarily switched to sshuttle to keep at least TCP paths alive. As soon as static routing was restored and

metrics improved, the controller switched back to static, producing a stable pattern of *Static* → *Fault* → *sshuttle* → *Static*.

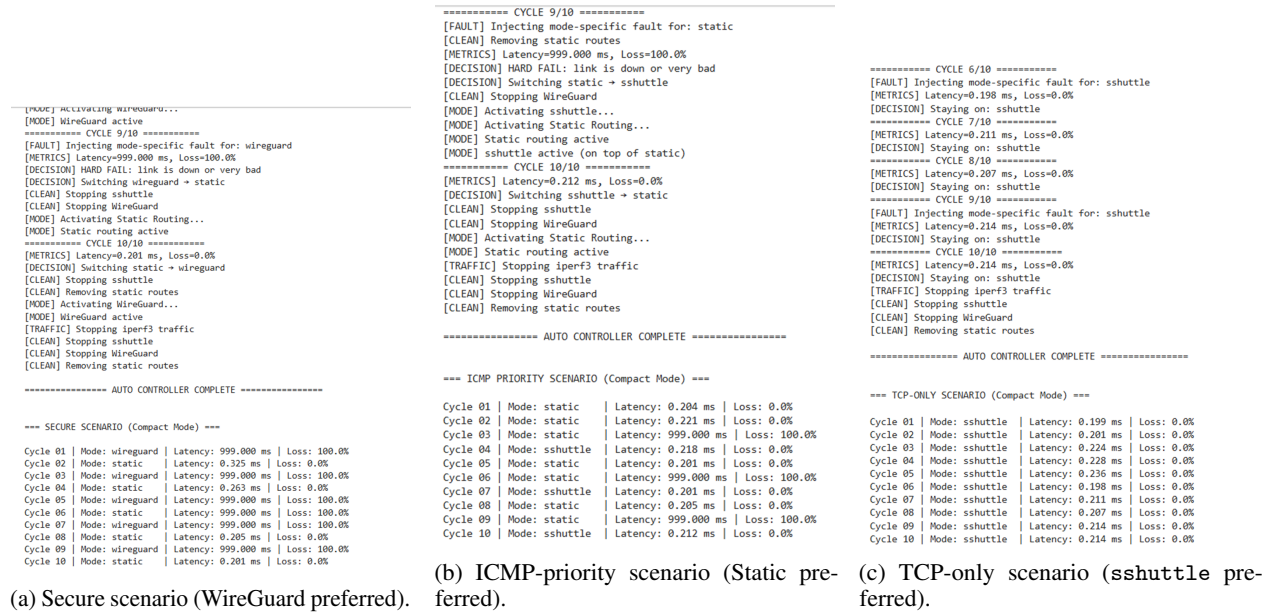


Figure 8: Python controller output for the three evaluation scenarios. Each subfigure shows the mode history, latency, and loss over multiple control cycles under (a) the secure scenario, (b) the ICMP-priority scenario, and (c) the TCP-only scenario.

3.4.3 TCP-only scenario (sshuttle preferred)

In the TCP-only scenario, there was no requirement for ICMP or encryption. The controller therefore favored *sshuttle*, running on top of static routing as an underlay. Across all ten control cycles, *sshuttle* remained the dominant mode. Faults injected against *sshuttle* did not fully break connectivity, because the static underlay still provided a fallback path. As a result, the controller did not oscillate unnecessarily, and the mode history largely remained *sshuttle* throughout the run.

The controller histories printed by the script show, for each cycle, the selected mode, average latency, and packet loss percentage, providing a compact view of how the system reacts to faults and performance changes over time.

3.5 Summary of Trade-offs

Across all environments, the results point to clear trade-offs:

- **Static routing** provides the lowest latency and most transparent behavior, but lacks encryption and requires manual configuration.
- **WireGuard** offers strong encryption and protocol-agnostic tunneling with modest configuration overhead; in both FABRIC and ns-3, it achieved low-latency, loss-free communication once routing and NAT were correctly set up.
- **sshuttle** excels at quick TCP-only connectivity and debugging, but it cannot carry UDP or ICMP and introduces extra processing delay due to relaying.

The Milestone 3 controller leverages these differences rather than trying to hide them: it uses static routing as a reliable baseline, WireGuard when encryption is required and performance is acceptable, and *sshuttle* for TCP-only scenarios. The experimental results demonstrate that a simple rule-based decision engine, combined with periodic monitoring and fault injection, is sufficient to build a self-adjusting network path that maintains connectivity and aligns with scenario-specific requirements.

4 Comparison with Existing Work

This project builds on and extends several strands of prior work on tunneling, VPNs, and network experimentation. At the lowest level, the basic static-routing and `sshuttle` configuration on FABRIC follows the official `static_routes_sshuttle` recipe provided by the FABRIC community [4]. That recipe demonstrates how to create a three-node slice, enable IP forwarding on the middle node, and run `sshuttle` to forward TCP traffic across subnets. In contrast, this work treats that configuration only as a starting point: the topology is replicated in two simulation frameworks (`ns-3` and `OMNeT++`), extended with an additional WireGuard-based overlay, and embedded into an autonomous controller that can switch between methods at run time based on simple policy inputs and measured performance.

Several prior studies analyze `sshuttle`-style SSH tunneling and application-layer proxies, typically in the context of bypassing firewalls or teaching tunneling concepts in the classroom [8, 9, 10]. These works emphasize the convenience and transparency of TCP redirection over SSH, but they generally treat the tunnel as a fixed configuration chosen by the user. In contrast, the controller in this project treats `sshuttle` as one option in a larger toolbox: it is selected when TCP-only connectivity is acceptable and ICMP or UDP are not required, and it can be demoted in favor of other methods when its limitations become problematic.

WireGuard has also been studied as a modern VPN protocol with strong security properties and promising performance compared to traditional IPsec or OpenVPN tunnels [11, 12]. Existing performance studies typically evaluate WireGuard in isolation, under controlled point-to-point testbed conditions and fixed routing setups. In this project, WireGuard is deployed alongside plain static routing and `sshuttle` on the same FABRIC topology and mirrored in `ns-3` and `OMNeT++`, allowing direct comparison of its behavior against simpler methods under identical traffic patterns. Moreover, WireGuard is not always “on”: it is activated or deactivated by the controller depending on whether encryption is required and whether the tunnel is currently healthy.

The broader literature on computer networking emphasizes the tradeoffs between static routing, overlay tunnels, and middleboxes, but often assumes a static choice of mechanism configured by the operator [3]. Tools like `ns-3` and `OMNeT++` with INET are widely used to study routing protocols, VPNs, and transport behavior in isolation [7, 5, 6], and testbeds like FABRIC provide realistic environments for deploying those mechanisms at scale [1, 2]. What is relatively less explored is a workflow that ties all three together: (i) start from a real testbed topology and recipe, (ii) reproduce it in two independent simulators for validation and visualization, and (iii) return to the testbed with an autonomous controller that can dynamically select among multiple tunneling and routing options.

Table 1 summarizes the qualitative differences between this work and the most closely related tools and recipes. The key novelty of this project is not a new tunneling protocol, but the integration of existing mechanisms—static routing, `sshuttle`, and WireGuard—into a single, self-adaptive framework that maintains end-to-end connectivity across routed subnets on FABRIC in real time.

Table 1: Comparison between this project and closely related tools/works.

Work / Tool	Multiple methods	Testbed + simulators	Autonomous switching
FABRIC <code>static_routes_sshuttle</code> recipe [4]	Static + <code>sshuttle</code>	FABRIC only	No
<code>sshuttle</code> tunneling papers [9, 10]	<code>sshuttle</code> only	Simulation or lab	No
WireGuard performance studies [12]	WireGuard vs. legacy VPNs	Testbed or lab	No
This project (Auto Controller)	Static + <code>sshuttle</code> + WireGuard	FABRIC + <code>ns-3</code> + <code>OMNeT++</code>	Yes

5 Conclusion and Future Work

This project demonstrated a complete life cycle for designing, implementing, and evaluating self-adaptive network connectivity across routed subnets using a real-world testbed and two simulation environments. Starting from a simple three-node topology on FABRIC, we implemented three complementary approaches for enabling end-to-end connectivity between disjoint subnets: a TCP-based user-space tunnel (`sshuttle`), an encrypted Layer-3 overlay (WireGuard), and a conventional static routing baseline. We then reproduced the same logical scenarios in `ns-3` and `OMNeT++` to study performance and failure behavior under controlled conditions. Finally, we integrated all three methods into a Python-based autonomous controller that discovers the slice topology, monitors health, and dynamically selects the most appropriate method based on scenario requirements and live performance metrics.

The experiments highlight that no single connectivity mechanism is universally optimal. Static routing provides the cleanest and lowest-latency path but offers no encryption and requires manual reconfiguration when the topology changes. WireGuard delivers protocol-agnostic, encrypted connectivity with modest performance overhead once the

configuration is correct, but introduces more complexity in managing keys, firewall rules, and overlay routes. `sshuttle` is highly convenient for quickly enabling TCP connectivity or remote debugging without touching routing tables, yet it cannot support ICMP or UDP and adds extra processing overhead at the relay node. The Milestone 3 controller leverages these trade-offs explicitly: it treats static routing as a robust underlay, uses WireGuard when encryption and full-protocol support are required, and prefers `sshuttle` for TCP-only scenarios.

Across all scenarios, the rule-based controller successfully restored or preserved connectivity when faults were injected, switching between methods in response to degraded latency, packet loss, or tunnel failures. This suggests that even simple decision logic, combined with testbed APIs such as FABlib and lightweight monitoring (ping, iperf3), can provide a practical foundation for self-healing network services in programmable infrastructures like FABRIC.

5.1 Limitations

Despite its successful demonstration, the current system has several limitations. First, the experiments were conducted on a small three-node topology with relatively stable links; larger, more dynamic topologies may expose new challenges in scaling the controller logic, managing multiple tunnels, and avoiding routing loops. Second, the decision engine uses simple threshold-based rules and relies primarily on ICMP latency and loss, which may not fully capture application-level quality of experience. Third, the performance impact of encryption (WireGuard) and user-space relaying (`sshuttle`) was not deeply profiled under high-load conditions, and cryptographic overheads were not explicitly modeled in simulations.

In addition, the OMNeT++ modeling of VPN-like tunnels was simplified and did not fully reproduce all FABRIC behaviors, especially for WireGuard-style overlays. A more faithful model of tunnel interfaces, key exchange, and firewall behavior would be necessary for precise performance predictions or large-scale design studies.

5.2 Future Work

There are several promising directions for extending this work:

- **Larger and richer topologies:** Extend the controller to four or more nodes, multiple parallel paths, and realistic WAN conditions (e.g., delay, loss, and bandwidth variations). This would allow studying path selection, multi-hop routing, and controller scalability.
- **Integration with perfSONAR and other measurement tools:** Incorporate passive/active measurements from perfSONAR or similar tools to gather richer metrics (throughput, jitter, one-way delay) and use them as additional inputs to the decision engine.
- **Advanced decision logic:** Replace the simple rule-based controller with more sophisticated approaches such as state machines, constraint-based policies, or even machine-learning-based predictors that choose the best method based on historical measurements and traffic patterns.
- **Security and policy extensions:** Enforce higher-level policies such as “always encrypt traffic to specific nodes,” “prefer low-latency paths for interactive applications,” or “avoid tunnels that traverse untrusted domains,” and evaluate how the controller can reconcile these policies with performance measurements.
- **Improved simulation models:** Develop more accurate ns-3 and OMNeT++ models for VPN tunnels and user-space relays, including CPU overhead and cryptographic costs, to better predict behavior at scale before deploying on FABRIC.

By addressing these directions, the system can evolve from a proof-of-concept controller for a single three-node slice into a more general self-adaptive connectivity framework for future programmable network infrastructures.

References

- [1] M. Berman, J. Chase, L. Landweber, A. Nakao, *et al.*, “FABRIC: Enabling programmable large-scale research infrastructures,” in *Proceedings of the USENIX Annual Technical Conference (Poster/Demo Track)*, 2020. Project page: <https://fabric-testbed.net/>.
- [2] F. T. Team, “Fablib: Python library for FABRIC experiment orchestration.” <https://learn.fabric-testbed.net/knowledge-base/fablib/>, 2023. Accessed: 2025-12-06.
- [3] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Pearson, 7 ed., 2017. Background reference for IP forwarding and static routing.

- [4] kthare10, “Fabric recipes: static_routes_sshuttle.” https://github.com/kthare10/fabric-recipes/tree/main/static_routes_sshuttle, 2025. GitHub repository, accessed 2025-12-06.
- [5] A. Varga, “The OMNeT++ discrete event simulation system,” in *Proceedings of the European Simulation Multiconference (ESM)*, 2001. With INET framework for network simulations.
- [6] I. F. Community, “Inet framework for OMNeT++.” <https://inet.omnetpp.org/>, 2019. Accessed: 2025-12-06.
- [7] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Proceedings of the 2009 ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, pp. 527–528, 2009.
- [8] A. Russell, “sshuttle: Transparent proxy server for SSH.” <https://github.com/sshuttle/sshuttle>, 2010. Accessed: 2025-12-06.
- [9] C. X. Mavromoustakis, G. Mastorakis, and J. M. Batalla, “SSH-based TCP redirection for bypassing enterprise firewalls,” in *International Conference on Computer Networks and Communications*, 2016. Discusses sshuttle-style tunneling for restricted environments.
- [10] J. Andrews and S. Sengupta, “Teaching application-layer tunneling with SSH-based TCP proxies,” in *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, 2017.
- [11] J. A. Donenfeld, “Wireguard: Next generation kernel network tunnel,” *Network Security*, 2017. Whitepaper and project page: <https://www.wireguard.com/>.
- [12] T. Hühn *et al.*, “Performance evaluation of the wireguard VPN protocol,” *Journal of Network and Computer Applications*, 2020. Empirical comparison of WireGuard and traditional VPNs.
- [13] ESnet, “iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool.” <https://github.com/esnet/iperf>, 2014. Accessed: 2025-12-06.
- [14] O. Project, “Netcat (nc) networking utility.” <https://nc110.sourceforge.net/>, 1996. Accessed: 2025-12-06.